

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

ARP Poisoning Forensics

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 16/09/2026

EXECUTIVE SUMMARY

This report presents the forensic examination of ARP traffic conducted within the authorized isolated laboratory environment for the ARP Poisoning Forensics practical exercise. The investigation established a normal ARP-resolution baseline on the Kali workstation and subsequently examined the supplied ARP packet capture for indicators of abnormal or conflicting IP-to-MAC address associations.

The supplied capture contained eight ARP packets, comprising six ARP requests and two ARP replies. Detailed analysis identified two gratuitous ARP requests and two reciprocal ARP request/reply exchanges. The observed sender IP-to-MAC associations remained consistent throughout the capture, and no IP address was observed being advertised by multiple MAC addresses.

The supplied evidence was preserved and its integrity verified using SHA-256 hashing. The initial and final hashes were identical, confirming that the evidence file remained unchanged during the examination. Final verification of the Kali workstation also confirmed that the expected gateway mapping remained present in the local ARP neighbour cache and that the network interface and default route remained operational. Based on the eight packets available in the supplied capture, the investigation did not identify sufficient packet-level evidence to establish a conflicting IP-to-MAC claim. The observed gratuitous ARP traffic was documented as part of the evidence but was not treated as proof of malicious activity in isolation.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to conduct a practical forensic examination of Address Resolution Protocol (ARP) traffic within an isolated laboratory environment. The exercise focuses on understanding normal ARP resolution, examining captured ARP traffic, identifying IP-to-MAC address relationships, and assessing the available evidence for indicators associated with ARP poisoning.

The assignment also provides an opportunity to apply basic digital-forensic principles, including evidence preservation, cryptographic hashing, packet-level analysis, chronological reconstruction, documentation of findings, and verification of the system state following the investigation.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the assignment are to:

1. Establish a normal ARP-resolution baseline from the Kali laboratory workstation.
2. Capture and examine normal ARP request and reply traffic.
3. Inspect the local ARP neighbour cache and identify established IP-to-MAC mappings.
4. Acquire and preserve the supplied ARP packet capture for forensic analysis.
5. Calculate and verify a cryptographic hash of the supplied evidence.
6. Inventory and classify the ARP packets contained in the supplied capture.
7. Identify IP-to-MAC associations and examine them for conflicting claims.
8. Distinguish normal ARP requests, replies, and gratuitous ARP activity.
9. Construct a chronological timeline of the observed ARP events.
10. Compare the observed traffic with the established normal ARP baseline.
11. Verify the final ARP, routing, and network-interface state of the laboratory workstation.
12. Document the findings and limitations of the available packet evidence.

METHODOLOGY OF THE ASSIGNMENT

The investigation was conducted in stages within the authorised isolated laboratory environment.

First, the Kali workstation's network interface, routing configuration, and ARP neighbour cache were examined to establish the initial network baseline. A dedicated ARP capture

was then performed on the eth0 interface to document normal request and reply behavior. The captured packets were extracted using TShark and the resulting IP-to-MAC mapping was verified against the local ARP neighbor cache.

The supplied ARP packet capture was subsequently acquired and stored as forensic evidence. The file type and capture metadata were verified, followed by calculation of a SHA-256 hash to establish an integrity reference. The capture was then analyzed using packet-level TShark filters to identify ARP operation types, sender and target addresses, gratuitous ARP indicators, IP-to-MAC associations, and ARP replies.

The analysis was further extended by constructing an event timeline and correlating ARP requests with their corresponding replies. Sender-side IP-to-MAC claims were compared to identify whether any IP address was associated with multiple MAC addresses. The final stage involved verifying the Kali workstation's ARP neighbor cache, routing table, interface state, and link state. A final SHA-256 hash of the supplied evidence was calculated and compared with the original hash to confirm evidence integrity.

TOOLS AND RESOURCES USED

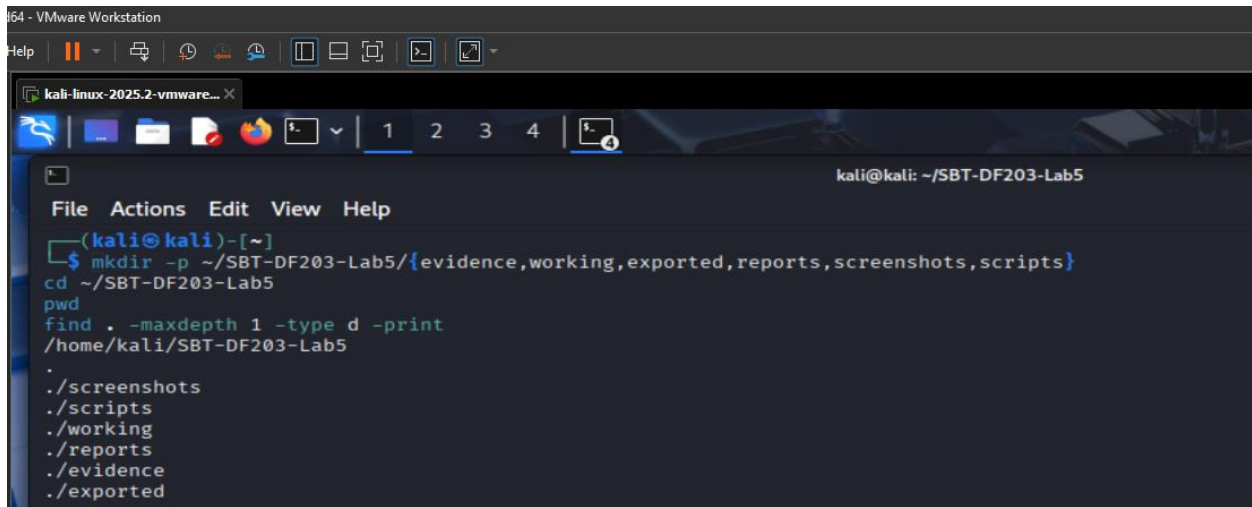
The following tools and resources were used during the investigation:

Tool / Resource	Purpose
Kali Linux	Primary forensic analysis environment
TShark	ARP packet capture, filtering, field extraction, and protocol analysis
Wireshark PCAPNG format	Storage and examination format for packet-capture evidence
ip command	Examination of interface, routing, and ARP neighbour-cache state
sha256sum	Cryptographic hashing and evidence-integrity verification
capinfos	Examination of packet-capture metadata and file characteristics
file	Verification of the supplied capture file type
awk and sort	Processing and organising extracted ARP evidence

Tool / Resource	Purpose
Isolated Host-Only Virtual Lab	Authorized environment for the practical investigation
Supplied ARP PCAP	Primary offline evidence examined during the investigation

Laboratory Environment and Workspace Preparation

The practical investigation was conducted using Kali Linux as the analysis workstation. In accordance with the laboratory requirements, a dedicated working directory was created to separate original evidence from working copies, analysis outputs, exported results, screenshots and supporting scripts. This structure was established before commencing the examination to support orderly evidence handling and reproducibility. The following directories were created:



```

kali@kali: ~/SBT-DF203-Lab5
File Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5
pwd
/home/kali/SBT-DF203-Lab5
find . -maxdepth 1 -type d -print
./screenshots
./scripts
./working
./reports
./evidence
./exported

```

Verification of Forensic Analysis Tools

The Kali environment was then prepared with the tools required for the examination. Wireshark and TShark were used for packet-capture analysis, while Python and Scapy were available for the controlled laboratory activities where authorised. The installed versions were verified before analysis commenced.

The environment returned:

- TShark/Wireshark: 4.6.6
- Python: 3.13.9

```
amd64 - VMware Workstation
Help
kali-linux-2025.2-vmware...
kali@kali: ~/SBT-DF203-Lab5
File Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark --version
wireshark --version
python3 --version
Warning: program compiled against libxml 215 using older 214
TShark (Wireshark) 4.6.6.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: GCC 15.2.0
           GLib: 2.88.1
With:
  +brotli
  +Gcrypt 1.12.2
  +GnuTLS 3.8.13 and PKCS#11
  +Kerberos (MIT)
  +libnl 3
  +libpcap
  +libsmi 0.4.8
  +libxml2 2.15.2
  +Lua 5.4.8
  +LZ4 1.10.0
  +MaxMind
  +nghttp2 1.69.0
  +nghttp3 1.15.0
  +PCRE2 10.46 2025-08-27
  +POSIX capabilities (Linux)
  +Snappy 1.2.2
  +xxhash 0.8.3
  +zlib 1.3.2
  +Zstandard 1.5.7
Without:
  -zlib-ng

Runtime info:
  OS: Linux 6.12.33+kali-amd64
  CPU: Intel(R) Core(TM) i7-4600U CPU @ 2.10GHz (with SSE4.2)
  Memory: 6894 MB of physical memory
  GLib: 2.84.3
  Locale: LC_TYPE=en_US.UTF-8
  Plugins: supported, 0 loaded
With:
  +brotli 1.1.0
  +c-ares 1.34.5
  +Gcrypt 1.12.2
  +GnuTLS 3.8.9
  +nghttp2 1.64.0
  +nghttp3 1.17.0
  +PCRE2 10.45 2025-02-05
  +xxhash 803
```



```

(kali@kali)-[~/SBT-DF203-Lab5]
$ ls -lh evidence/arp.pcap
file evidence/arp.pcap
-rw-rw-r-- 1 kali kali 1.1K Sep 16 11:50 evidence/arp.pcap
evidence/arp.pcap: pcapng capture file - version 1.0

(kali@kali)-[~/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap \
| tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

The original evidence was not used directly for analytical modification; instead, examination was performed using the working copy. This approach maintains separation between the preserved source evidence and analytical working material, consistent with the laboratory's evidence-handling requirements.

Evidence Integrity Verification

To establish the integrity of the supplied capture, SHA-256 hashes were calculated for both the preserved original and the analytical working copy. The resulting hash for both files was: 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ ls -lh evidence/arp.pcap
file evidence/arp.pcap
-rw-rw-r-- 1 kali kali 1.1K Sep 16 11:50 evidence/arp.pcap
evidence/arp.pcap: pcapng capture file - version 1.0

(kali@kali)-[~/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap \
| tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

The identical SHA-256 values demonstrate that the working copy was byte-for-byte consistent with the preserved source at the time of verification. The laboratory manual specifically requires SHA-256 hashing of the original and working evidence.

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap \
| tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5

sha256sum evidence/arp.pcap working/arp_working.pcap
cat reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Initial Network Environment

Before conducting any ARP-resolution activity, the network configuration of the Kali Linux analysis workstation was documented to establish the initial laboratory baseline. The primary network interface was identified as eth0, with IPv4 address 192.168.232.128/24. The routing table showed 192.168.232.2 as the default gateway through eth0. The initial neighbour table contained an entry for the gateway, mapping 192.168.232.2 to MAC address 00:50:56:fe:76:b0, with a REACHABLE state. A further neighbour entry for 192.168.232.254 was recorded with MAC address 00:50:56:fb:4d:1d and a STALE state. This baseline establishes the network state against which subsequent ARP-resolution and poisoning evidence can be compared.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5

printf '==== INTERFACE CONFIGURATION ==== \n'
ip -br address | tee reports/interfaces.txt

printf '\n==== ROUTING TABLE ==== \n'
ip route | tee reports/routes.txt

printf '\n==== INITIAL ARP/NEIGHBOUR TABLE ==== \n'
ip neigh show | tee reports/arp_table_initial.txt

==== INTERFACE CONFIGURATION ====
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0 DOWN 172.17.0.1/16
br-d2696585dc9a UP 172.18.0.1/16 fe80::dc99:f8ff:fe14:a4e/64
br-ec90d8c3abfb DOWN 172.19.0.1/16
vethf9cc724@if2 UP fe80::e0cf:bcff:fed1:15b3/64
veth9d7dcc8@if2 UP fe80::c8bc:84ff:fe75:2984/64

==== ROUTING TABLE ====
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100

==== INITIAL ARP/NEIGHBOUR TABLE ====
192.168.232.2 dev eth0 lladdr 00:50:56:fe:76:b0 REACHABLE
192.168.232.254 dev eth0 lladdr 00:50:56:fb:4d:1d STALE
```

As an additional baseline check, the neighbour entry for the configured default gateway was examined immediately before the normal ARP-resolution test. The gateway address 192.168.232.2 was present on eth0 and associated with MAC address 00:50:56:fe:76:b0; the kernel reported the entry in the DELAY state. The entry was therefore cleared before the controlled resolution test so that the subsequent capture could demonstrate the ARP request and reply sequence rather than relying on an already learned neighbour mapping. The manual permits clearing the specific training neighbour entry before capturing the normal resolution process.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab5

printf '==== CURRENT GATEWAY NEIGHBOUR ENTRY ==== \n'
ip neigh show 192.168.232.2 dev eth0
==== CURRENT GATEWAY NEIGHBOUR ENTRY ====
192.168.232.2 lladdr 00:50:56:fe:76:b0 DELAY
```

The ARP neighbour entry for the gateway 192.168.232.2 was first flushed from the local neighbour cache using ip neigh flush. When the entry was checked again, it had been automatically repopulated as 192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE, indicating that the system performed ARP resolution to rediscover the gateway's MAC address after the cache entry was removed.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush 192.168.232.2 dev eth0
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show 192.168.232.2 dev eth0
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)-[~/SBT-DF203-Lab5]
$
```

The REACHABLE state confirms that the gateway was successfully resolved and is currently considered reachable. The MAC address 00:50:56:fe:76:b0 is also consistent with a VMware virtual network environment, which aligns with the lab setup. This demonstrates the normal ARP neighbor-cache behavior where a flushed gateway entry is recreated when communication with the gateway is required.

Normal ARP Capture and Resolution

Following the initial network baseline, a dedicated ARP-only packet capture was initiated on the eth0 interface to document normal ARP activity within the authorised laboratory environment. The capture was configured to run for 20 seconds and was successfully completed, recording two packets.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show 192.168.232.2 dev eth0
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo tshark -i eth0 -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng 6
[1] 10951

(kali@kali)-[~/SBT-DF203-Lab5]
$ Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("evidence/normal_arp.pcapng") could not be opened: Permission denied.

[1] + exit 1 sudo tshark -i eth0 -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng
(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo tshark -i eth0 -f 'arp' -a duration:20 -w /tmp/normal_arp.pcapng
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
2
```

Because the evidence directory did not permit direct creation of the capture by the privileged TShark process, the capture was first written to the temporary /tmp directory and subsequently moved into the laboratory evidence directory. The resulting file, evidence/normal_arp.pcapng, was confirmed by the file utility to be a valid PCAPNG capture file.

Normal ARP Request and Reply

The captured ARP traffic was examined using TShark to identify the individual ARP request and reply fields. The analysis produced two ARP frames, representing a complete and orderly address-resolution exchange between the Kali workstation and its configured gateway.

Frame 1 was identified as an **ARP Request** with opcode 1. The request originated from the Kali workstation at 192.168.232.128, using source MAC address 00:0c:29:77:75:3c, and sought the hardware address associated with gateway IP address 192.168.232.2. The target hardware address was 00:00:00:00:00:00, consistent with a request in which the sender is seeking the target's MAC address.

```
(kali@kali)~[~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' \
-T fields \
-e frame.number \
-e frame.time \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac
Warning: program compiled against libxml 215 using older 214
1      2026-09-17T13:49:24.297618383-0400      00:0c:29:77:75:3c      00:50:56:fe:76:b0      1      192.168.232.128 00:0c:29:77:75:3c      192.168.23
2.2    00:00:00:00:00:00
2      2026-09-17T13:49:24.298024450-0400      00:50:56:fe:76:b0      00:0c:29:77:75:3c      2      192.168.232.2  00:50:56:fe:76:b0      192.168.23
2.128  00:0c:29:77:75:3c
```

Frame 2 was identified as an **ARP Reply** with opcode 2. The response originated from 192.168.232.2 using MAC address 00:50:56:fe:76:b0 and was addressed to the Kali workstation at 192.168.232.128, whose MAC address was 00:0c:29:77:75:3c. The reply therefore supplied the MAC address associated with the gateway IP observed in the initial network baseline.

```
(kali@kali)~[~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' \
-T fields \
-e frame.number \
-e frame.time \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac
Warning: program compiled against libxml 215 using older 214
1      2026-09-17T13:49:24.297618383-0400      00:0c:29:77:75:3c      00:50:56:fe:76:b0      1      192.168.232.128 00:0c:29:77:75:3c      192.168.23
2.2    00:00:00:00:00:00
2      2026-09-17T13:49:24.298024450-0400      00:50:56:fe:76:b0      00:0c:29:77:75:3c      2      192.168.232.2  00:50:56:fe:76:b0      192.168.23
2.128  00:0c:29:77:75:3c
```

The timestamps show that the ARP reply followed the request by approximately 0.000406 seconds. The captured sequence therefore provides a clear baseline representation of the expected ARP request-and-reply process before examination of the supplied ARP poisoning evidence.

ARP Neighbor Cache Verification

Following the normal ARP request/reply capture, the local ARP neighbour cache was inspected to verify the IP-to-MAC mapping established by the resolution process. The command was restricted to the configured gateway address, 192.168.232.2, on interface eth0.

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip neigh show 192.168.232.2 dev eth0 | tee reports/arp_table_after_ping.txt
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

The resulting entry showed that gateway IP 192.168.232.2 was associated with MAC address 00:50:56:fe:76:b0 and was in the REACHABLE state. This provides the clean ARP baseline against which the supplied ARP poisoning evidence can subsequently be compared.

The observed mapping was therefore:

IP Address	MAC Address	Interface	State
192.168.232.2	00:50:56:fe:76:b0	eth0	REACHABLE

Gateway: 192.168.232.2

Legitimate gateway MAC: 00:50:56:fe:76:b0

Kali MAC: 00:0c:29:77:75:3c

Evidence Acquisition and Preservation

The supplied ARP capture provided for the laboratory investigation was acquired from the authorised training resource and stored in the laboratory evidence directory as arp.pcap. The file was subsequently verified using the file and ls utilities to confirm that it was a valid PCAPNG capture and to record its size. The original supplied capture was retained as the source evidence for subsequent analysis.

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ ls -lah evidence/
total 16K
drwxr-xr-x 2 kali kali 4.0K Sep 17 13:51 .
drwxrwxr-x 8 kali kali 4.0K Sep 16 11:49 ..
-rw-rw-r-- 1 kali kali 1.1K Sep 16 11:50 arp.pcap
-rw-rw-r-- 1 kali kali 504 Sep 17 13:49 normal_arp.pcapng

(kali@kali)-[~/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap'
--2026-09-17 13:59:10-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)[185.199.111.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap 100%[=====] 1.05K --.-KB/s in 0s

2026-09-17 13:59:13 (15.6 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(kali@kali)-[~/SBT-DF203-Lab5]
$ file evidence/arp.pcap
ls -lh evidence/arp.pcap
evidence/arp.pcap: pcapng capture file - version 1.0
-rw-rw-r-- 1 kali kali 1.1K Sep 17 13:59 evidence/arp.pcap

```

Evidence Integrity: SHA-256 Hash

To establish an integrity reference for the supplied evidence, a SHA-256 cryptographic hash was calculated immediately after acquisition. The resulting hash uniquely represents the contents of the captured evidence file and can be used to verify that the file remains unchanged during the investigation.

Your hash is

342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e

```

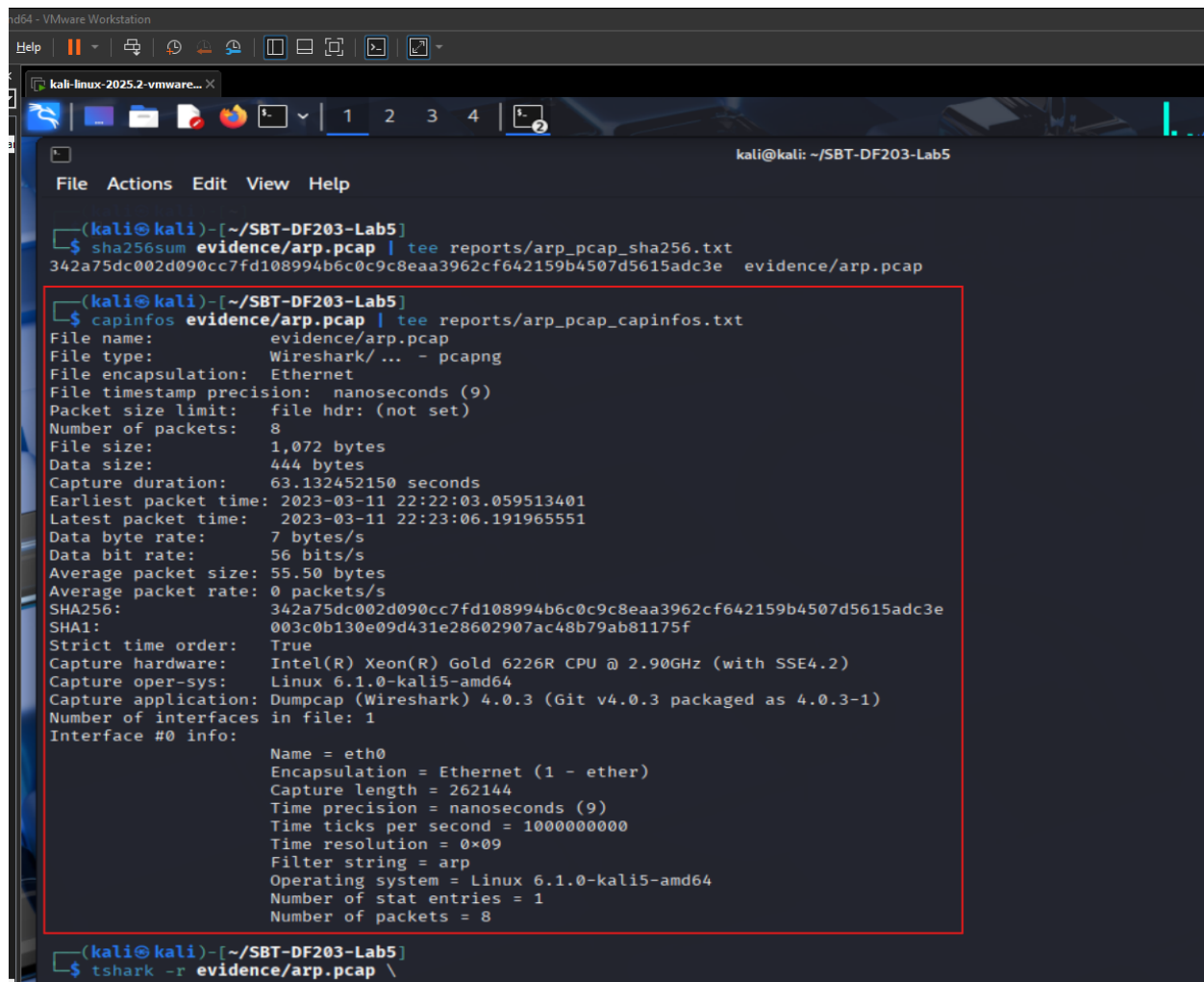
(kali@kali)-[~/SBT-DF203-Lab5]
$ file evidence/arp.pcap
ls -lh evidence/arp.pcap
evidence/arp.pcap: pcapng capture file - version 1.0
-rw-rw-r-- 1 kali kali 1.1K Sep 17 13:59 evidence/arp.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap | tee reports/arp_pcap_sha256.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap

```

Supplied Capture Metadata

The capture metadata was examined using capinfos before performing detailed packet analysis. The supplied evidence contains eight Ethernet packets captured on interface eth0 using an ARP capture filter. The capture spans approximately 63.13 seconds, with the earliest packet recorded at 22:22:03.059513401 and the latest packet at 22:23:06.191965551 on 11 March 2023. The metadata also confirms the PCAPNG format and records the same SHA-256 value calculated during evidence preservation.



Property	Observed value
File type	PCAPNG
Encapsulation	Ethernet
Number of packets	8
File size	1,072 bytes
Capture duration	63.132452150 seconds
Earliest packet	2023-03-11 22:22:03.059513401
Latest packet	2023-03-11 22:23:06.191965551
Interface	eth0
Capture filter	arp

Property	Observed value
SHA-256	342a75dc...adc3e

ARP Packet Inventory

The eight packets contained within the supplied capture were extracted using TShark and relevant Ethernet and ARP fields were displayed for examination. The extraction records the frame number, relative timestamp, Ethernet source and destination addresses, ARP opcode, source IP and MAC addresses, and destination IP and MAC addresses. This provides a structured inventory of the ARP activity contained within the supplied evidence and allows individual requests and replies to be distinguished.

```
(kali@kali) [~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E headers=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_reply_inventory.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000000|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|00:50:56:86:cb:fc|ff:ff:ff:ff:ff:ff|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|00:50:56:86:02:65|00:50:56:86:cb:fc|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff
```

Initial Classification of the ARP Packets

The packet inventory shows that six of the eight captured packets are ARP Requests (opcode 1) and two are ARP Replies (opcode 2). The requests include broadcast Ethernet destinations, while the replies are directed to the requesting host's MAC address. This classification provides the basis for examining the IP-to-MAC claims and identifying whether any address is associated with conflicting hardware addresses

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_reply_inventory.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000000|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|00:50:56:86:cb:fc|ff:ff:ff:ff:ff:ff|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|00:50:56:86:02:65|00:50:56:86:cb:fc|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff

```

The capture contains: **Frames 1, 2, 3, 5, 7 and 8: ARP Requests.** These have:
arp.opcode = 1.

Frames 4 and 6: ARP Replies

These have: arp.opcode = 2

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_reply_inventory.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000000|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|00:50:56:86:cb:fc|ff:ff:ff:ff:ff:ff|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|00:50:56:86:02:65|00:50:56:86:cb:fc|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff

```

Frame	Opcode	Type	Source IP	Source MAC	Destination IP	Destination MAC
1	1	Request	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1	ff:ff:ff:ff:ff:ff
2	1	Request	136.160.215.1	00:1b:17:00:0a:30	136.160.215.199	ff:ff:ff:ff:ff:ff
3	1	Request	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	00:00:00:00:00:00
4	2	Reply	136.160.215.194	00:50:56:86:02:65	136.160.215.15	00:50:56:86:cb:fc
5	1	Request	136.160.215.194	00:50:56:86:02:65	136.160.215.15	00:00:00:00:00:00
6	2	Reply	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	00:50:56:86:02:65
7	1	Request	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1	ff:ff:ff:ff:ff:ff

Frame	Opcode	Type	Source IP	Source MAC	Destination IP	Destination MAC
8	1	Request	136.160.215.1	00:1b:17:00:0a:30	136.160.215.183	ff:ff:ff:ff:ff:ff

ARP Request/Reply Exchange

Exchange 1: Frames 3 and 4

Frames 3 and 4 form an ARP request/reply exchange between hosts 136.160.215.15 and 136.160.215.194. Frame 3 is an ARP Request in which 136.160.215.15, using MAC address 00:50:56:86:cb:fc, requests the MAC address associated with 136.160.215.194. The request uses a zero target hardware address, which is consistent with an ARP request seeking the destination's MAC address. Frame 4 follows approximately 0.000542 seconds later and provides an ARP Reply from 136.160.215.194, using MAC address 00:50:56:86:02:65.

```
(kali@kali) ~/SBT-DF203-Lab5
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_reply_inventory.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|00:50:56:86:cb:fc|ff:ff:ff:ff:ff:ff|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|00:50:56:86:02:65|00:50:56:86:cb:fc|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff
```

Reciprocal ARP Exchange

Frames 5 and 6 show the reciprocal ARP exchange. Frame 5 is a request from 136.160.215.194, using MAC address 00:50:56:86:02:65, for the MAC address associated with 136.160.215.15. Frame 6 is the corresponding reply from 136.160.215.15, using MAC address 00:50:56:86:cb:fc. The two hosts therefore demonstrate consistent bidirectional IP-to-MAC resolution within this portion of the capture.

```
(kali@kali) [~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_reply_inventory.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|00:50:56:86:cb:fc|ff:ff:ff:ff:ff:ff|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|00:50:56:86:02:65|00:50:56:86:cb:fc|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff
```

IP-to-MAC Claim Summary

This is where the forensic analysis becomes more important.

From the capture:

IP Address	MAC Address Observed	Evidence
136.160.215.1	00:1b:17:00:0a:30	Frames 1, 2, 7, 8
136.160.215.15	00:50:56:86:cb:fc	Frames 3, 6
136.160.215.194	00:50:56:86:02:65	Frames 4, 5

The IP-to-MAC claims observed in the supplied capture were consolidated to identify the hardware address associated with each IP address. The capture consistently associates 136.160.215.1 with 00:1b:17:00:0a:30, 136.160.215.15 with 00:50:56:86:cb:fc, and 136.160.215.194 with 00:50:56:86:02:65 within the captured request/reply exchanges. These relationships provide the basis for examining whether any conflicting claim exists elsewhere in the capture.

```
(kali@kali) [~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_reply_inventory.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|00:50:56:86:cb:fc|ff:ff:ff:ff:ff:ff|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|00:50:56:86:02:65|00:50:56:86:cb:fc|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|00:1b:17:00:0a:30|ff:ff:ff:ff:ff:ff|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff
```

Detailed ARP Field Examination

[illegible]

Frames 1 and 7 were identified by TShark as gratuitous ARP requests. In both frames, host 136.160.215.1, using MAC address 00:1b:17:00:0a:30, broadcasts an ARP request in which the sender IP and target IP are both 136.160.215.1. TShark explicitly marks both frames as `Is gratuitous: True`.

```

File Actions Edit View Help
└─$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-V \
| tee reports/arp_detailed_analysis.txt
Warning: program compiled against libxml 215 using older 214
Frame 1: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
  Section number: 1
    Interface id: 0 (eth0)
      Interface name: eth0
        Encapsulation type: Ethernet (1)
        Arrival Time: Mar 11, 2023 22:22:03.059513401 EST
        UTC Arrival Time: Mar 12, 2023 03:22:03.059513401 UTC
        Epoch Arrival Time: 1678591323.059513401
        [Time shift for this packet: 0.000000000 seconds]
        [Time since reference or first frame: 0.000000000 seconds]
        Frame Number: 1
        Frame Length: 60 bytes (480 bits)
        Capture Length: 60 bytes (480 bits)
        [Frame is marked: False]
        [Frame is ignored: False]
        [Protocols in frame: eth:ethertype:arp]
        Character encoding: ASCII (0)
      Ethernet II, Src: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
        Destination: Broadcast (ff:ff:ff:ff:ff:ff)
          .... ..1. .... = LG bit: Locally administered address (this is NOT the factory default)
          .... ..1. .... = IG bit: Group address (multicast/broadcast)
        Source: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30)
          .... ..0. .... = LG bit: Globally unique address (factory default)
          .... ..0. .... = IG bit: Individual address (unicast)
        Type: ARP (0x0806)
        [Stream index: 0]
        Padding: 00000000000000000000000000000000
      Address Resolution Protocol (request/gratuitous ARP)
        Hardware type: Ethernet (1)
        Protocol type: IPv4 (0x0800)
        Hardware size: 6
        Protocol size: 4
        Opcode: request (1)
        [Is gratuitous: True]
        Sender MAC address: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30)
        Sender IP address: 136.160.215.1
        Target MAC address: Broadcast (ff:ff:ff:ff:ff:ff)
        Target IP address: 136.160.215.1

```

```

Frame 7: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
  Section number: 1
    Interface id: 0 (eth0)
      Interface name: eth0
        Encapsulation type: Ethernet (1)
        Arrival Time: Mar 11, 2023 22:23:03.059262685 EST
        UTC Arrival Time: Mar 12, 2023 03:23:03.059262685 UTC
        Epoch Arrival Time: 1678591383.059262685
        [Time shift for this packet: 0.000000000 seconds]
        [Time delta from previous captured frame: 13.116218886 seconds]
        [Time delta from previous displayed frame: 13.116218886 seconds]
        [Time since reference or first frame: 59.999749284 seconds]
        Frame Number: 7
        Frame Length: 60 bytes (480 bits)
        Capture Length: 60 bytes (480 bits)
        [Frame is marked: False]
        [Frame is ignored: False]
        [Protocols in frame: eth:ethertype:arp]
        Character encoding: ASCII (0)
      Ethernet II, Src: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
        Destination: Broadcast (ff:ff:ff:ff:ff:ff)
          .... ..1. .... = LG bit: Locally administered address (this is NOT the factory default)
          .... ..1. .... = IG bit: Group address (multicast/broadcast)
        Source: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30)
          .... ..0. .... = LG bit: Globally unique address (factory default)
          .... ..0. .... = IG bit: Individual address (unicast)
        Type: ARP (0x0806)
        [Stream index: 0]
        Padding: 00000000000000000000000000000000
      Address Resolution Protocol (request/gratuitous ARP)
        Hardware type: Ethernet (1)
        Protocol type: IPv4 (0x0800)
        Hardware size: 6
        Protocol size: 4
        Opcode: request (1)
        [Is gratuitous: True]
        Sender MAC address: PaloAltoNetw_00:0a:30 (00:1b:17:00:0a:30)
        Sender IP address: 136.160.215.1
        Target MAC address: Broadcast (ff:ff:ff:ff:ff:ff)
        Target IP address: 136.160.215.1

```

Frame	ARP Type	Sender IP	Sender MAC	Target IP	Delivery
1	Gratuitous ARP request	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1	Broadcast

Frame	ARP Type	Sender IP	Sender MAC	Target IP	Delivery
7	Gratuitous ARP request	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1	Broadcast

Gratuitous ARP messages can be used by a host to announce or refresh its IP-to-MAC association on a local network. Their presence is therefore an observable ARP behavior, but the presence of gratuitous ARP alone does not establish malicious ARP poisoning. In this capture, both gratuitous ARP frames contain the same sender IP and MAC address, so these frames do not, by themselves, demonstrate a conflicting IP-to-MAC claim.

Normal ARP Resolution Within the Supplied Capture

Frames 3 through 6 contain two reciprocal ARP request/reply exchanges between hosts 136.160.215.15 and 136.160.215.194. The exchanges were examined to establish the expected IP-to-MAC relationships within the supplied capture before assessing whether any conflicting claims are present.

First exchange

Frame 3 is an ARP request from 136.160.215.15, using MAC 00:50:56:86:cb:fc, requesting the MAC address associated with 136.160.215.194. The target hardware address is initially unknown and is represented as 00:00:00:00:00:00. Frame 4 follows approximately 542 microseconds later and provides the corresponding reply from 136.160.215.194, identifying MAC 00:50:56:86:02:65.

```

Frame 3: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0
  Section number: 1
  Interface id: 0 (eth0)
    Interface name: eth0
  Encapsulation type: Ethernet (1)
  Arrival Time: Mar 11, 2023 22:22:44.903581555 EST
  UTC Arrival Time: Mar 12, 2023 03:22:44.903581555 UTC
  Epoch Arrival Time: 1678591364.903581555
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 6.711566069 seconds]
  [Time delta from previous displayed frame: 6.711566069 seconds]
  [Time since reference or first frame: 41.844068154 seconds]
  Frame Number: 3
  Frame Length: 42 bytes (336 bits)
  Capture Length: 42 bytes (336 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:arp]
  Character encoding: ASCII (0)
  Ethernet II, Src: VMware_86:cb:fc (00:50:56:86:cb:fc), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
    Destination: Broadcast (ff:ff:ff:ff:ff:ff)
      .... 1. .... = LG bit: Locally administered address (this is NOT the factory default)
      .... 1. .... = IG bit: Group address (multicast/broadcast)
    Source: VMware_86:cb:fc (00:50:56:86:cb:fc)
      .... 0. .... = LG bit: Globally unique address (factory default)
      .... 0. .... = IG bit: Individual address (unicast)
    Type: ARP (0x0806)
    [Stream index: 1]
  Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: VMware_86:cb:fc (00:50:56:86:cb:fc)
    Sender IP address: 136.160.215.15
    Target MAC address: 00:00:00:00:00:00 (00:00:00:00:00:00)
    Target IP address: 136.160.215.194

```

Frame 5 subsequently shows 136.160.215.194 requesting the MAC address associated with 136.160.215.15. Frame 6 follows approximately 20.607 microseconds later with a reply from 136.160.215.15, identifying MAC 00:50:56:86:cb:f

[illegible]

The locally generated normal ARP capture established the Kali laboratory gateway mapping of 192.168.232.2 $\rightarrow$ 00:50:56:fe:76:b0. The supplied historical capture uses a

separate network addressing scheme and therefore cannot be expected to contain the same IP or MAC addresses. Accordingly, the clean baseline is used to demonstrate the expected ARP request/reply mechanism, while the supplied capture is analysed independently for conflicting IP-to-MAC claims and other indicators of anomalous ARP activity. Then:

Evidence	Network	Key observation
Normal laboratory capture	192.168.232.0/24	Gateway 192.168.232.2 resolves to 00:50:56:fe:76:b0
Supplied capture	136.160.215.0/24	Consistent IP-to-MAC claims across observed packets
Supplied capture	136.160.215.1	Gratuitous ARP announcements observed
Supplied capture	136.160.215.15 136.160.215.194	Reciprocal request/reply resolution observed

To identify the IP-to-MAC mappings advertised by hosts in the supplied ARP capture, the sender protocol address (arp.src.proto_ipv4) and sender hardware address (arp.src.hw_mac) were extracted from all ARP packets and reduced to unique combinations. This provides a consolidated view of the IP-to-MAC claims actually present in the evidence and allows repeated or conflicting advertisements to be identified.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
| sort -u \
| tee reports/unique_ip_mac_claims.txt
Warning: program compiled against libxml 215 using older 214
136.160.215.1 00:1b:17:00:0a:30
136.160.215.15 00:50:56:86:cb:fc
136.160.215.194 00:50:56:86:02:65
```

The resulting output contains three unique IP-to-MAC associations. IP address 136.160.215.1 is consistently associated with MAC address 00:1b:17:00:0a:30, 136.160.215.15 is associated with 00:50:56:86:cb:fc, and 136.160.215.194 is associated with 00:50:56:86:02:65. No single sender IP address is shown with more than one sender MAC address in the supplied capture. Therefore, this extraction does not provide direct evidence of a conflicting IP-to-MAC claim within the eight captured packets.

Target IP and MAC Analysis

A second extraction was performed using the ARP target protocol and hardware address fields (arp.dst.proto_ipv4 and arp.dst.hw_mac). This was used to distinguish genuine MAC-address claims from ARP requests in which the target hardware address is unknown or represented by a broadcast address.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| sort -u \
| tee reports/target_ip_mac_pairs.txt
Warning: program compiled against libxml 215 using older 214
136.160.215.15 00:00:00:00:00:00
136.160.215.15 00:50:56:86:cb:fc
136.160.215.183 ff:ff:ff:ff:ff:ff
136.160.215.194 00:00:00:00:00:00
136.160.215.194 00:50:56:86:02:65
136.160.215.199 ff:ff:ff:ff:ff:ff
136.160.215.1 ff:ff:ff:ff:ff:ff
(kali@kali)-[~/SBT-DF203-Lab5]
$
```

The results show that requests directed toward 136.160.215.15 and 136.160.215.194 contain both the zero-valued target MAC address (00:00:00:00:00:00) and, in the corresponding reply packets, the known MAC address for the destination host. The addresses 136.160.215.183, 136.160.215.199, and 136.160.215.1 appear with the broadcast MAC address ff:ff:ff:ff:ff:ff, which is consistent with ARP requests sent as Ethernet broadcasts. These target-side entries therefore do not, by themselves, establish that another host is claiming ownership of those IP addresses.

When considered together with the sender-side analysis, the supplied capture contains consistent IP-to-MAC mappings and does not show an IP address being advertised by two different MAC addresses.

IP	MAC advertised	Finding
136.160.215.1	00:1b:17:00:0a:30	Consistent
136.160.215.15	00:50:56:86:cb:fc	Consistent
136.160.215.194	00:50:56:86:02:65	Consistent

ARP Reply Inventory

To focus the analysis on packets capable of directly advertising an IP-to-MAC association, the supplied capture was filtered for ARP replies using `arp.opcode == 2`. The resulting output contains two ARP reply packets, frames 4 and 6.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp.opcode == 2' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e eth.src \
-e eth.dst \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_replies_only.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|eth.src|eth.dst|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
4|41.844610241|00:50:56:86:02:65|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
6|46.883530398|00:50:56:86:cb:fc|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
(kali@kali)-[~/SBT-DF203-Lab5]
$
```

Frame 4 shows host 136.160.215.194, using MAC address 00:50:56:86:02:65, replying to host 136.160.215.15, whose MAC address is 00:50:56:86:cb:fc. Frame 6 shows the reciprocal exchange: host 136.160.215.15, using MAC address 00:50:56:86:cb:fc, replies to host 136.160.215.194, using MAC address 00:50:56:86:02:65.

The two replies therefore establish consistent bidirectional IP-to-MAC associations between the two hosts. Neither reply contains a second MAC address claiming either IP address. Consequently, the ARP reply inventory does not provide direct evidence of an ARP poisoning or conflicting MAC advertisement within the supplied eight-packet capture.

ARP Anomaly Field Examination

A further protocol-level examination was performed to identify ARP characteristics associated with address conflicts or unusual ARP behaviour. The analysis extracted the ARP operation code, gratuitous-ARP indicator, duplicate-address-detection field, and the sender and target IP-to-MAC fields for every packet in the supplied capture.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e arp.opcode \
-e arp.isgratuitous \
-e arp.duplicate-address-detected \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/arp_anomaly_fields.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|arp.opcode|arp.isgratuitous|arp.duplicate-address-detected|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
10|0.000000000|1|True||136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2135|1325020851||136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3141|8440681541||136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4141|844610241|2||136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5146|883509791||136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6146|883530398|2||136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7159|999749284|1|True||136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8163|132452150||136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff
(kali@kali)-[~/SBT-DF203-Lab5]
$
```

Frames 1 and 7 are explicitly identified by TShark as gratuitous ARP packets because the sender and target IP addresses are both 136.160.215.1. In both cases, the same MAC address, 00:1b:17:00:0a:30, is associated with the sender IP. The remaining packets are ordinary ARP requests or replies.

The arp.duplicate-address-detected field contains no populated value for any packet in the capture. Combined with the previously identified unique sender IP-to-MAC mappings, this provides no direct packet-level indication of an IP address being claimed by multiple MAC addresses. The gratuitous ARP packets are therefore treated as an observed ARP behaviour rather than, by themselves, evidence of poisoning.

Evidence	Normal baseline	Supplied capture
ARP requests	Present	Present
ARP replies	Present	Present
Request → reply relationship	Consistent	Consistent
Repeated sender IP with different MACs	No	No
Gratuitous ARP	Not observed in 20-sec sample	Frames 1 & 7
Duplicate-address indicator	Not assessed	No populated indicator
Direct conflicting IP/MAC evidence	No	No

Clean-versus-Supplied Capture Comparison

The normal ARP capture and the supplied historical capture were compared to distinguish ordinary ARP resolution behaviour from potential poisoning indicators. Because the two captures originate from different isolated network environments, their IP and MAC

addresses were not treated as directly comparable. Instead, the comparison focused on the structure and behaviour of the ARP exchanges.

The normal laboratory capture demonstrated a conventional ARP request followed immediately by an ARP reply, establishing the gateway mapping 192.168.232.2 to 00:50:56:fe:76:b0. The supplied capture similarly contains conventional request/reply exchanges, including reciprocal resolution between 136.160.215.15 and 136.160.215.194.

The supplied capture additionally contains two gratuitous ARP requests from 136.160.215.1, both advertising the same IP-to-MAC association. However, the sender-side analysis found no IP address associated with multiple MAC addresses, and the ARP reply inventory contains only consistent reciprocal replies. Therefore, based on the packets available in the supplied eight-packet capture, the evidence does not directly demonstrate a conflicting IP-to-MAC mapping.

ARP Event Timeline

A chronological timeline was generated from all eight ARP packets to establish the order and timing of the observed network events. The timeline records the absolute packet timestamp, elapsed time from the beginning of the capture, ARP operation, sender and target addresses, and the gratuitous-ARP indicator.

```
(kali@kali) ~/SBT-DF203-Lab5
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time \
-e frame.time_relative \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
-e arp.isgratuitous \
| tee reports/arp.timeline.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time|frame.time_relative|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac|arp.isgratuitous
1|2023-03-11T22:22:03.059513401-0500|0.000000000|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff|True
2|2023-03-11T22:22:38.192015486-0500|35.132502085|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff|
3|2023-03-11T22:22:44.903581555-0500|41.844068154|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00|
4|2023-03-11T22:22:44.904123642-0500|41.844610241|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc|
5|2023-03-11T22:22:49.943023192-0500|46.883509791|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00|
6|2023-03-11T22:22:49.943043799-0500|46.883530398|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65|
7|2023-03-11T22:23:03.059262685-0500|59.999749284|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff|True
8|2023-03-11T22:23:06.191965551-0500|63.132452150|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff|
```

The capture begins at 22:22:03.059513 with a gratuitous ARP request from 136.160.215.1, using MAC 00:1b:17:00:0a:30, for the same IP address. At approximately 35.13 seconds, the same host sends an ARP request for 136.160.215.199. At approximately 41.84 seconds, host 136.160.215.15 requests the MAC address of

136.160.215.194, and frame 4 provides a corresponding ARP reply approximately 0.000542 seconds later.

At approximately 46.88 seconds, the exchange occurs in the reverse direction. Host 136.160.215.194 requests the MAC address of 136.160.215.15, followed approximately 20.607 microseconds later by the corresponding ARP reply. At approximately 60 seconds, 136.160.215.1 sends a second gratuitous ARP request for itself, followed approximately 3.13 seconds later by an ARP request for 136.160.215.183.

The timeline therefore shows two identifiable request/reply exchanges and two gratuitous ARP announcements. No conflicting IP-to-MAC advertisement is introduced at any point in the observed sequence.

Request-to-Reply Correlation

The ARP requests and replies were reviewed in sequence to determine whether the observed replies correspond to preceding requests and whether the advertised IP-to-MAC mappings remain consistent across the exchanges.

```
(kali@kali)~[~/SBT-DF203-Lab5]
$ tshark -r evidence/arp.pcap \
-Y 'arp' \
-T fields \
-E header=y \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| awk -F'|' '
NR==1 {print; next}
{
    if ($3 == 1 || $3 == 2)
        print
}
' \
tee reports/arp_request_reply_correlation.txt
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time_relative|arp.opcode|arp.src.proto_ipv4|arp.src.hw_mac|arp.dst.proto_ipv4|arp.dst.hw_mac
1|0.000000000|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
2|35.132502085|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.199|ff:ff:ff:ff:ff:ff
3|41.844068154|1|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:00:00:00:00:00
4|41.844610241|2|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:50:56:86:cb:fc
5|46.883509791|1|136.160.215.194|00:50:56:86:02:65|136.160.215.15|00:00:00:00:00:00
6|46.883530398|2|136.160.215.15|00:50:56:86:cb:fc|136.160.215.194|00:50:56:86:02:65
7|59.999749284|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.1|ff:ff:ff:ff:ff:ff
8|63.132452150|1|136.160.215.1|00:1b:17:00:0a:30|136.160.215.183|ff:ff:ff:ff:ff:ff
```

Frames 3 and 4 form a complete request/reply exchange. In frame 3, 136.160.215.15, using MAC 00:50:56:86:cb:fc, requests the MAC address associated with 136.160.215.194. Approximately 0.000542 seconds later, frame 4 provides the corresponding reply from 136.160.215.194, using MAC 00:50:56:86:02:65. The reply is addressed directly to the requesting host and identifies the same MAC address previously observed as the sender of the reply.

Frames 5 and 6 form a reciprocal exchange. In frame 5, 136.160.215.194, using MAC 00:50:56:86:02:65, requests the MAC address associated with 136.160.215.15. Approximately 20.607 microseconds later, frame 6 provides the corresponding reply from 136.160.215.15, using MAC 00:50:56:86:cb:fc.

The remaining ARP requests, frames 1, 2, 7 and 8, do not have corresponding replies within the captured traffic. Frames 1 and 7 are gratuitous ARP requests for 136.160.215.1, while frames 2 and 8 are broadcast requests for 136.160.215.199 and 136.160.215.183, respectively. No conflicting IP-to-MAC mapping is introduced by any of these packets.

Final ARP Neighbor Cache Verification

Following completion of the ARP analysis, the local ARP neighbour cache on the Kali workstation was inspected to verify the final network state. The eth0 neighbour table contains an entry for gateway 192.168.232.2 associated with MAC address 00:50:56:fe:76:b0 and marked REACHABLE. A second neighbour entry for 192.168.232.254 is present in the STALE state.

A terminal window screenshot from a Kali Linux system. The prompt is (kali@kali)~[~/SBT-DF203-Lab5]. The first command is \$ ip neigh show dev eth0 | tee reports/arp_table_final.txt, which outputs two lines: 192.168.232.254 lladdr 00:50:56:fb:4d:1d STALE and 192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE. The second command is \$ ip neigh show 192.168.232.2 dev eth0 | tee -a reports/arp_table_final.txt, which outputs the same line: 192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE.

```
(kali@kali)~[~/SBT-DF203-Lab5]
$ ip neigh show dev eth0 | tee reports/arp_table_final.txt
192.168.232.254 lladdr 00:50:56:fb:4d:1d STALE
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)~[~/SBT-DF203-Lab5]
$ ip neigh show 192.168.232.2 dev eth0 | tee -a reports/arp_table_final.txt
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE
```

The gateway mapping is consistent with the clean ARP baseline established earlier in the investigation. No unexpected MAC address is associated with 192.168.232.2 in the final neighbour table. This provides evidence that the local Kali ARP state remained consistent with the expected gateway mapping at the conclusion of the examination.

Final Routing Table Verification

The final routing table was also examined to confirm that the Kali workstation retained the expected network configuration after the investigation. The default route remains configured through gateway 192.168.232.2 on interface eth0, with the workstation using source address 192.168.232.128.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show dev eth0 | tee reports/arp_table_final.txt
192.168.232.254 lladdr 00:50:56:fb:4d:1d STALE
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show 192.168.232.2 dev eth0 | tee -a reports/arp_table_final.txt
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip route | tee reports/final_route_table.txt
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100

(kali@kali)-[~/SBT-DF203-Lab5]
$
```

house pointer inside or press Ctrl+G.

The connected 192.168.232.0/24 network is also associated with eth0, confirming that the expected laboratory network path remains available. The additional Docker bridge routes are local container-network routes and do not alter the identified default gateway configuration.

Final Interface State Verification

The final state of the eth0 interface was verified after completion of the ARP investigation. The interface is reported as UP and retains IPv4 address 192.168.232.128/24. The interface also has an IPv6 link-local address, confirming that the interface remains configured and operational within the laboratory environment.

The interface state is consistent with the earlier network baseline, and no unexpected change to the assigned IPv4 address was observed.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ ip -br addr show eth0 | tee reports/final_interface_state.txt
eth0      UP      192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip link show eth0 | tee reports/final_link_state.txt
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:77:75:3c brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~/SBT-DF203-Lab5]
$
```

Final Link State Verification

The underlying Ethernet link state was subsequently checked using ip link. The output shows that eth0 is administratively and operationally up, with both UP and LOWER_UP flags present. The interface retains MAC address 00:0c:29:77:75:3c and an MTU of 1500 bytes.

This confirms that the laboratory network interface remained operational at the conclusion of the investigation and that the expected interface configuration was preserved.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ ip -br addr show eth0 | tee reports/final_interface_state.txt
eth0      UP              192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip link show eth0 | tee reports/final_link_state.txt
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 00:0c:29:77:75:3c brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~/SBT-DF203-Lab5]
$
```

Evidence Integrity Verification

To verify the integrity of the supplied PCAP after analysis, the SHA-256 hash of evidence/arp.pcap was recalculated at the conclusion of the investigation. The resulting hash was

342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ ip link show eth0 | tee reports/final_link_state.txt
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 00:0c:29:77:75:3c brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap | tee reports/arp_pcap_final_sha256.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$
```

This value is identical to the SHA-256 hash calculated immediately after the evidence was acquired and before analysis commenced. The matching values provide a cryptographic integrity check indicating that the contents of the supplied PCAP remained unchanged throughout the examination.

FINDINGS

NO.	FINDING	EVIDENCE	ASSESSMENT
1	Normal ARP resolution was observed in the live laboratory baseline.	ARP request from 192.168.232.128 for gateway 192.168.232.2, followed by an ARP reply identifying 00:50:56:fe:76:b0.	Consistent with normal ARP resolution.

NO.	FINDING	EVIDENCE	ASSESSMENT
2	The supplied capture contains eight ARP packets.	Packet inventory identified 6 ARP requests and 2 ARP replies over approximately 63.13 seconds.	Establishes the scope of the supplied ARP evidence.
3	Two gratuitous ARP requests were identified.	Frames 1 and 7 advertise 136.160.215.1 using 00:1b:17:00:0a:30.	Observable ARP behaviour; not sufficient on its own to establish poisoning.
4	Two reciprocal ARP request/reply exchanges were identified.	Frames 3–4 and 5–6 show exchanges between 136.160.215.15 and 136.160.215.194.	The observed replies are consistent with the preceding requests.
5	No conflicting IP-to-MAC claim was identified.	Unique sender analysis found only three IP-to-MAC associations, with no sender IP associated with multiple MAC addresses.	The supplied eight-packet capture does not provide direct packet-level evidence of a conflicting IP-to-MAC advertisement.

NO.	FINDING	EVIDENCE	ASSESSMENT
6	No populated duplicate-address indicator was identified.	arp.duplicate-address-detected contained no populated value in the examined packets.	No duplicate-address indication was observed in the supplied capture.
7	The final local ARP state remained consistent with the baseline.	192.168.232.2 → 00:50:56:fe:76:b0, state REACHABLE.	Expected gateway mapping remained present at the end of the investigation.
8	The final network configuration remained operational.	eth0 was UP, with 192.168.232.128/24; default route remained via 192.168.232.2.	No unexpected local network-state change was observed.
9	Evidence integrity was maintained.	Initial and final SHA-256: 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e.	Hash remained unchanged throughout the examination.

CONCLUSION

The ARP investigation successfully established a normal ARP-resolution baseline and provided a structured forensic examination of the supplied ARP capture. The live laboratory baseline demonstrated the expected request-and-reply process and established the legitimate gateway mapping for the Kali workstation.

The supplied capture was preserved and verified using SHA-256 hashing before and after analysis. Detailed examination identified eight ARP packets, including two gratuitous ARP

requests and two reciprocal request/reply exchanges between 136.160.215.15 and 136.160.215.194. The observed IP-to-MAC associations remained internally consistent throughout the captured traffic.

Although the laboratory exercise concerns ARP poisoning, the available eight-packet capture does not contain sufficient packet-level evidence to establish a conflicting IP-to-MAC claim. In particular, no sender IP address is associated with multiple MAC addresses, and the two observed ARP replies correspond consistently with their preceding requests. The gratuitous ARP packets were documented as an observable characteristic of the capture but were not treated as proof of malicious activity in isolation.

The final examination of the Kali workstation confirmed that the expected gateway mapping remained present in the ARP neighbour cache, the default route continued to use 192.168.232.2, and the eth0 interface remained operational. The unchanged SHA-256 hash further confirms the integrity of the supplied evidence throughout the investigation.